

Unit - III

Linked list

Linked lists are the most common data structures. They are referred to as an array of connected objects where data is stored in the pointer fields. Linked lists are useful when the number of elements to be stored in a list is indefinite.

Concept of Linked Lists An array is represented in memory using sequential mapping, which has the property that elements are fixed distance apart. But this has the following disadvantage. It makes insertion or deletion at any arbitrary position in an array a costly operation, because this involves the movement of some of the existing elements.

Therefore, a linked list is a list of elements in which the elements of the list can be placed anywhere in memory, and these elements are linked with each other using an explicit link field, that is, by storing the address of the next element in the link field of the previous element. This program uses a strategy of inserting a node in an existing list to get the list created. An insert function is used for this. The insert function takes a pointer to an existing list as the first parameter, and a data value with which the new node is to be created as a second parameter, creates a new node by using the data value, appends it to the end of the list, and returns a pointer to the first node of the list. Initially the list is empty, so the pointer to the starting node is NULL.

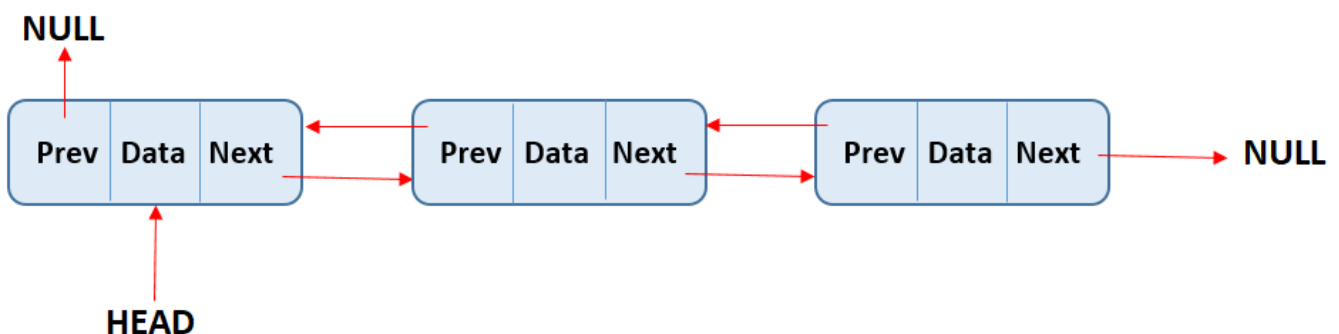
Therefore, when insert is called first time, the new node created by the insert becomes the start node. Subsequently, the insert traverses the list to get the pointer to the last node of the existing list, and puts the address of the newly created node in the link field of the last node, thereby appending the new node to the existing list. The main function reads the value of the number of nodes in the list. Calls iterate that many times by going in a while loop to create the links with the specified number of nodes.

2.5 Types of linked list

Single linked list Double linked list Circular linked list

A doubly-linked list is a linked data structure that consists of a set of sequentially linked records called nodes. Each node contains two fields, called links, that are references to the previous and to the next node in the sequence of nodes. In the single linked list each node provides information about where the next node is in the list. It faces difficulty if we are pointing to a specific node, then we can move only in the direction of the links. It has no idea about where the previous node lies in memory. The only way to find the node which precedes that specific node is to start back at the beginning of the list. The same problem arises when one wishes to delete an arbitrary node from a single linked list. Since in order to easily delete an arbitrary node one must know the preceding node. This problem can be avoided by using Doubly Linked List, we can store in each node not only the address of next node but also the address of the previous node in the linked list. A node in Doubly Linked List has three fields:

1. Data
2. Previous Link
3. Next Link



Implementation of Doubly Linked List

Structure of a node of Doubly Linked List can be defined as:

```
struct node
{
    int data;
    struct node *llink;
    struct node *rlink;
}
```

Circular Linked List

Circular Linked List is another remedy for the drawbacks of the Single Linked List besides Doubly Linked List. A slight change to the structure of a linear list is made to convert it to circular linked list; link field in the last node contains a pointer back to the first node rather than a Null.

Representation of Linked List

Because each node of an element contains two parts, we have to represent each node through a structure.

While defining linked list we must have recursive definitions:

```
struct node
{
    int data;
    struct node * link;
}
```

Here, link is a pointer of struct node type i.e. it can hold the address of variable of struct node type. Pointers permit the referencing of structures in a uniform way, regardless of the organization of the structure being referenced. Pointers are capable of representing a much more complex relationship between elements of a structure than a linear order.

Initialization:

```
main()
{
    struct node *p, *list, *temp;
    list = p = temp = NULL;
    .
    .
    .
}
```



Example:

Program:

```
# include <stdio.h>
# include <stdlib.h>

struct node
{
    int data;
    struct node *link;
```

```

};
struct node *insert(struct node *p, int n)
{
    struct node *temp;
    /* if the existing list is empty then insert a new node as the
    starting node */
    if(p==NULL)
    {
        p=(struct node *)malloc(sizeof(struct node)); /* creates new
        node data value passes
        as parameter */
        if(p==NULL)
        {
            printf("Error\n");
            exit(0);
        }
        p-> data = n;
        p-> link = p; /* makes the pointer pointing to itself because
        it is a circular list*/
    }
    else
    {
        temp = p;
        /* traverses the existing list to get the pointer to the last node
        of it */
        while (temp->link != p)
        temp = temp->link;
        temp-> link = (struct node *)malloc(sizeof(struct node)); /*
        creates new node using
        data value passes as
        parameter and puts its
        address in the link field
        of last node of the
        existing list*/
        if(temp -> link == NULL)
        {
            printf("Error\n");
            exit(0);
        }
        temp = temp->link;
    }
}

```

```

temp-> data = n;
temp-> link = p;
}
return (p);
}
void printlist( struct node *p )
{
struct node *temp;
temp = p;
printf("The data values in the list are\n");
if(p!= NULL)
{
do
{
printf("%d\t",temp->data);
temp=temp->link;
} while (temp!= p);
}
else
printf("The list is empty\n");
}
void main()
{
int n;
int x;
struct node *start = NULL ;
printf("Enter the nodes to be created \n");
scanf("%d",&n);
while ( n -- > 0 )
{
printf( "Enter the data values to be placed in a node\n");
scanf("%d",&x);
start = insert ( start, x);
}
printf("The created list is\n");
printlist( start );
}

```

Deleting the Specified Node in Singly Linked List

To delete a node, first we determine the node number to be deleted (this is based on the assumption that the nodes of the list are numbered serially from 1 to n). The list is then traversed to get a pointer to the node whose number is given, as well as a pointer to a node that appears before the

node to be deleted. Then the link field of the node that appears before the node to be deleted is made to point to the node that appears after the node to be deleted, and the node to be deleted is freed.



Lab Exercise:

```
# include <stdio.h>
# include <stdlib.h>

struct node *delet( struct node *, int );
int length ( struct node * );

struct node
{
    int data;
    struct node *link;
};

struct node *insert(struct node *p, int n)
{
    struct node *temp;
    if(p==NULL)
    {
        p=(struct node *)malloc(sizeof(struct node));
        if(p==NULL)
        {
            printf("Error\n");
            exit(0);
        }
        p-> data = n;
        p-> link = NULL;
    }
    else
    {
        temp = p;
        while (temp->link != NULL)
            temp = temp->link;
        temp-> link = (struct node *)malloc(sizeof(struct node));
        if(temp -> link == NULL)
        {
            printf("Error\n");
            exit(0);
        }
        temp = temp->link;
        temp-> data = n;
        temp-> link = NULL;
    }
    return (p);
}
```

```

}
void printlist( struct node *p )
{
printf("The data values in the list are\n");
while (p!= NULL)
{
printf("%d\t",p-> data);
p = p->link;
}
}
void main()
{
int n;
int x;
struct node *start = NULL;
printf("Enter the nodes to be created \n");
scanf("%d",&n);
while ( n- > 0 )
{
printf( "Enter the data values to be placed in a node\n");
scanf("%d",&x);
start = insert ( start, x );
}
printf(" The list before deletion id\n");
printlist( start );
printf("% \n Enter the node no \n");
scanf( " %d",&n);
start = delet (start , n );
printf(" The list after deletion is\n");
printlist( start );
}
/* a function to delete the specified node*/
struct node *delet( struct node *p, int node_no )
{
struct node *prev, *curr ;
int i;
if (p == NULL )
{
printf("There is no node to be deleted \n");
}
}

```

```

else
{
if ( node_no> length (p))
{
printf("Error\n");
}
else
{
prev = NULL;
curr = p;
i = 1 ;
while ( i<node_no )
{
prev = curr;
curr = curr->link;
i = i+1;
}
if ( prev == NULL )
{
p = curr ->link;
free ( curr );
}
else
{
prev -> link = curr ->link ;
free ( curr );
}
}
return(p);
}

/* a function to compute the length of a linked list */
int length ( struct node *p )
{
int count = 0 ;
while ( p != NULL )
{
count++;
p = p->link;
}
}

```



```
return ( count ) ;  
}
```

Inserting a Node after the Specified Node in a Singly Linked List

To insert a new node after the specified node, first we get the number of the node in an existing list after which the new node is to be inserted. This is based on the assumption that the nodes of the list are numbered serially from 1 to n. The list is then traversed to get a pointer to the node, whose number is given. If this pointer is x, then the link field of the new node is made to point to the node pointed to by x, and the link field of the node pointed to by x is made to point to the new node. Figures 2.3 and 2.4 show the list before and after the insertion of the node, respectively.

Insertion in Linked List can happen at following places:

At the beginning of the linked list.

At the end of the linked list.

At a given position in the linked list.

Algorithm: Insertion at beginning

Step 1: IF PTR = NULL

Write OVERFLOW

Go to Step 7

[END OF IF]

Step 2: SET NEW_NODE = PTR

Step 3: SET PTR = PTR → NEXT

Step 4: SET NEW_NODE → DATA = VAL

Step 5: SET NEW_NODE → NEXT = HEAD

Step 6: SET HEAD = NEW_NODE

Step 7: EXIT

Deletion from a Linked List

Delete from beginning

Delete from end

Delete from middle/ given position

Find the previous node of the node to be deleted.

Change the next pointer of the previous node

Free the memory of the deleted node.

In case of first node deletion, we need to update the head of the linked list.

Algorithm: Deletion at beginning

Step 1: IF HEAD = NULL

Write UNDERFLOW

Go to Step 5

[END OF IF]

Step 2: SET PTR = HEAD

Step 3: SET HEAD = HEAD → NEXT

Step 4: FREE PTR

Step 5: EXIT

Searching in linked list

Searching is performed to find the location of a particular element in the list. Traversing is performed in the list and make the comparison of every element of the list with the specified element. If the element is matched with any of the list element then the location of the element is returned from the function.

Algorithm: Searching in linked list

Step 1: SET PTR = HEAD

Step 2: Set I = 0

STEP 3: IF PTR = NULL

WRITE "EMPTY LIST"

GOTO STEP 8

END OF IF

STEP 4: REPEAT STEP 5 TO 7 UNTIL PTR != NULL

STEP 5: if ptr → data = item

write i+1

End of IF

STEP 6: I = I + 1

STEP 7: PTR = PTR → NEXT

[END OF LOOP]

STEP 8: EXIT

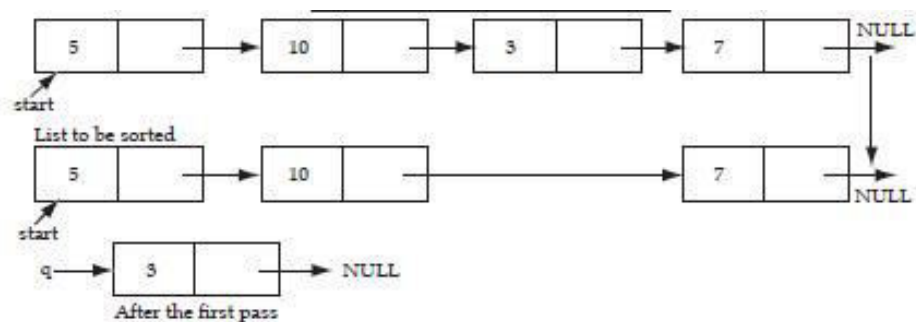
Linked List Common Errors

Here is summary of common errors of linked lists. Read these carefully, and read them again when you have problem that you need to solve.

1. Allocating a new node to step through the linked list; only a pointer variable is needed.
2. Confusing the and the -> operators.
3. Not setting the pointer from the last node to 0 (null).
4. Not considering special cases of inserting/removing at the beginning or the end of the linked list.
5. Applying the delete operator to a node (calling the operator on a pointer to the node) before it is removed. Delete should be done after all pointer manipulations are completed.
6. Pointer manipulations that are out of order. These can ruin the structure of the linked list.

Sorting and Reversing a Linked List

To sort a linked list, first we traverse the list searching for the node with a minimum data value. Then we remove that node and append it to another list which is initially empty. We repeat this process with the remaining list until the list becomes empty, and at the end, we return a pointer to the beginning of the list to which all the nodes are moved.



Sorting of linked list

To reverse a list, we maintain a pointer each to the previous and the next node, then we make the link field of the current node point to the previous, make the previous equal to the current, and the current equal to the next.

Arrays vs. Linked list

Array	Linked list
Data elements are stored in contiguous locations in memory.	New elements can be stored anywhere and a reference is created for the new element using pointers.
Insertion and Deletion operations are costlier since the memory locations are consecutive and fixed.	Insertion and Deletion operations are fast and easy in a linked list.
Memory is allocated during the compile time (<i>Static memory allocation</i>).	Memory is allocated during the run-time (<i>Dynamic memory allocation</i>).
Size of the array must be specified at the time of array declaration/initialization.	Size of a Linked list grows/shrinks as and when new elements are inserted/deleted.

Summary

- An array is a set of same data elements grouped together. Arrays can be one-dimensional or multidimensional.
- A linear or one-dimensional array is a structured collection of elements (often called as array elements) that are accessed individually by specifying the position of each element with a single index value.
- Multidimensional arrays are nothing but "arrays of arrays". Two subscripts are used to refer to the elements.
- The operations that are performed on an array are adding, sorting, searching, and traversing.
- Traversing an array refers to moving in inward and outward direction to access each element in an array.
- Linked list is a technique of dynamically implementing a list using pointers. A linked list contains two fields namely, data field and link field.
- A singly-linked list consists of only one pointer to point to another node and the last node always points to NULL to indicate the end of the list.
- A doubly-linked list consists of two pointers, one to point to the next node and the other to point to the previous node.
- In a circular singly-linked list, the last node always points to the first node to indicate the circular nature of the list.
- A circular doubly-linked list consists of two pointers for forward and backward traversal and the last node points to the first node.

Unit 04: Queues

CONTENTS

Objectives

Introduction

4.1 Fundamentals of Queues

4.2 Basic Operations of Queue

4.3 Types of Queue

4.4 Implementation of Queues

4.5 Applications of Queues

Summary

Keywords

Self Assessment

Answers for Self Assessment

Review Questions

Further Readings

Objectives

After studying this unit, you will be able to:

- Learn implementation of queues
- Explain priority queue
- Discuss applications of queues

Introduction

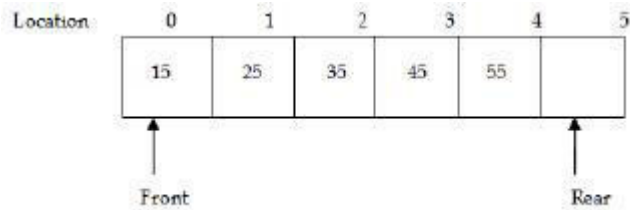
A queue is a linear list of elements that consists of two ends known as front and rear. We can delete elements from the front end and insert elements at the rear end of a queue. A queue in an application is used to maintain a list of items that are ordered not by their values but by their sequential value.

The queue abstract data type is also a widely used one with applications very common in real life. An example comes from the operating system software where the scheduler picks up the next process to be executed on the system from a queue data structure. In this unit, we would study the various properties of queues, their operations and implementation strategies.

4.1 Fundamentals of Queues

A queue is an ordered collection of items in which deletion takes place at one end, which is called the front of the queue, and insertion at the other end, which is called the rear of the queue. The queue is a 'First In First Out' system (FIFO). In a time-sharing system, there can be many tasks waiting in the queue, for access to disk storage or for using the CPU. The queues in a bank, or railway station counter are examples of queue. The first person in the queue is the first to be attended.

The two main operations in the queue are insertion and deletion of items. The queue has two pointers, the front pointer points to the first element of the queue and the rear pointer points to the last element of the queue.



4.2 Basic Operations of Queue

The basic operations of queue are insertion and deletion of items which are referred as enqueue and dequeue respectively. In enqueue operation, an item is added to the rear end of the queue. In dequeue operation, the item is deleted from the front end of the queue.

Insert at Rear End

To insert an item into the queue, first it should be verified whether the queue is full or not. If the queue is full, a new item cannot be inserted into the queue. The condition $FRONT = NULL$ indicates that the queue is empty. If the queue is not full, items are inserted at the rear end. When an item is added to the queue, the value of rear is incremented by 1.

In queue we need to maintain two data pointers, front and rear. Operations on queue are comparatively difficult to implement than that of stacks

Step 1 – Check if the queue is full.

Step 2 – If the queue is full, produce overflow error and exit.

Step 3 – If the queue is not full, increment rear pointer to point the next empty space.

Step 4 – Add data element to the queue location, where the rear is pointing.

Step 5 – return success.

Algorithm: Enqueue operation

```

procedure enqueue(data)
    if queue is full
        return overflow
    endif
    rear ← rear + 1
    queue[rear] ← data
    return true
end procedure
  
```

Delete from the Front End

To delete an item from the stack, first it should be verified that the queue is not empty. If the queue is not empty, the items are deleted at the front end of the queue. When an item is deleted from the queue, Dequeue operation include two tasks: access the data where front is pointing and remove the data after access.

Step 1 – Check if the queue is empty.

Step 2 – If the queue is empty, produce underflow error and exit.

Step 3 – If the queue is not empty, access the data where front is pointing.

Step 4 – Increment front pointer to point to the next available data element.

Step 5 – Return success. the value of the front is incremented by 1.

Algorithm: Dequeue operation

```

procedure dequeue
    if queue is empty
  
```

```

    return underflow
end if
data = queue[front]
front ← front + 1
return true
end procedure

```



Example:

```

/*Program of queue using array*/
/*insertion and deletion in a queue*/
/*insertion and deletion in a queue*/
# include <stdio.h>
# define MAX 50
int queue_arr[MAX];
int rear = -1;
int front = -1;
void ins_delete();
void insert();
void display();
void main()
{
int choice;
while(1)
{
printf("1.Insert\n");
printf("2.Delete\n");
printf("3.Display\n");
printf("4.Quit\n");
printf("Enter your choice : \n");
scanf("%d",&choice);
switch(choice)
{
case 1 : insert();
break;
case 2 :ins_delete();
break;
case 3: ins_display();
break;
case 4: exit(1);
default:

```

```

printf("Wrong choice\n");
}/*End of switch*/
}/*End of while*/
}/*End of main()*/
void insert()
{
int added_item;
if (rear==MAX-1)
printf("Queue overflow\n");
else
{
if (front==-1) /*If queue is initially empty */
front=0;
printf("Enter an element to add in the queue : ");
scanf("%d", &added_item);
rear=rear+1;
queue_arr[rear] = added_item ;
}
} /*End of insert()*/
void ins_delete()
{
if (front == -1 || front > rear)
{
printf("Queue underflow\n");
return ;
}
else
{
printf("Element deleted from queue is : %d\n", queue_arr[front]);
front=front+1;
}
} /*End of delete() */
void display()
{
int i;
if (front == -1)
printf("Queue is empty\n");
else
{
printf("Elements in the queue:\n");
for(i=front;i<= rear;i++)

```

```
printf("%d ",queue_arr[i]);
printf("\n");
}
} /*End of display() */
```

Output:

1. Insert
2. Delete
3. Display
4. Quit

Enter your choice: 1

Enter an element to add in the queue: 25

Enter your choice: 1

Enter an element to add in the queue: 36

Enter your choice: 3

Elements in the queue: 25, 36

Enter your choice: 2

Element deleted from the queue is: 25

In this example:

1. The preprocessor directives #include are given. MAXSIZE is defined as 50 using the #define statement.
2. The queue is declared as an array using the declaration int queue_arr[MAX].
3. In the while loop, the different options are displayed on the screen and the value entered in the variable choice is accepted.
4. The switch case compares the value entered and calls the method corresponding to it. If the value entered is invalid, it displays the message "Wrong choice".
5. Insert method: The insert method inserts item in the queue. The if condition checks whether the queue is full or not. If the queue is full, the "Queue overflow" message is displayed. If the queue is not full, the item is inserted in the queue and the rear is incremented by 1.
6. Delete method: The delete method deletes item from the queue. The if condition checks whether the queue is empty or not. If the queue is empty, the "Queue underflow" message is displayed. If the queue is not empty, the item is deleted and front is incremented by 1.
7. Display method: The display method displays the contents of the queue. The if condition checks whether the queue is empty or not. If the queue is not empty, it displays all the items in the queue.

4.3 Types of Queue

Simple Queue

In a simple queue, insertion takes place at the rear and removal occurs at the front. It follows the FIFO (First in First out) rule.

Circular Queue

In a circular queue, the last element points to the first element making a circular link. In a circular queue, the rear end is connected to the front end forming a circular loop. An advantage of circular queue is that, the insertion and deletion operations are independent of one another. This prevents an interrupt handler from performing an insertion operation at the same time when the main function is performing a deletion operation.

Double ended queue

Double ended queue is also known as deque. It is a type of queue where the insertions and deletions happen at the front or the rear end of the queue. The various operations that can be performed on the double ended queue are:

1. Insert an element at the front end
2. Insert an element at the rear end
3. Delete an element at the front end
4. Delete an element at the rear end



Example:

Program for Implementation of Circular Queue.

```
#include<stdio.h>
#include<conio.h>
#define SIZE 5
int Q_F(int COUNT)
{
    return (COUNT==SIZE)? 1:0;
}
int Q_E(int COUNT)
{
    return (COUNT==0)? 1:0;
}
void rear_insert(int item, int Q[], int *R, int *COUNT)
{
    if(Q_F(*COUNT))
    {
        printf("Queue overflow");
        return;
    }
    *R=(*R+1) % SIZE;
    Q[*R]=num;
    *COUNT+=1;
}
void front_delete(int Q[], int *F, int *COUNT)
{
    if(Q_E(*COUNT))
    {
        printf("Queue underflow");
        return;
    }
    printf("The deleted element is %d\n", Q[*F]);
    *F=(*F+1) % SIZE;
```

```

*COUNT-=1;
}
void display(int Q[], int F, int COUNT)
{
int i,j;
if(Q_E(COUNT))
{
printf("Queue is empty\n");
return;
}
printf("The contents of the queue are:\n");
i=F;
for(j=1;j<=COUNT; j++)
{
printf("%d\n", Q[i]);
i=(i+1) % SIZE;
}
printf("\n");
}
void main()
{
int choice, num, COUNT, F, R, Q[20];
clrscr();
F=0;
R=-1;
COUNT=0;
for(;;)
{
printf("1. Insert at front\n");
printf("2. Delete at rear end\n");
printf("3. Display\n");
printf("4. Exit\n");
scanf("%d", &choice);
switch(choice)
{
case 1: printf("Enter the number to be inserted\n");
scanf("%d", &num);
rear_insert(num, Q, &R, &COUNT);
break;
case 2: front_delete(Q, &F, &COUNT);
break;

```

```

case 3: display(Q, F, COUNT);
break;
default: exit(0);
}
}
}

```

Output:

```

1. Insert at rear end
2. Delete at front end
3. Display
4. Exit
1
Enter the number to be inserted
50
1. Insert at rear end
2. Delete at front end
3. Display
4. Exit
1
Enter the number to be inserted
60
1. Insert at rear end
2. Delete at front end
3. Display
4. Exit
3
The contents of the queue are 50 60
1. Insert at rear end
2. Delete at front end
3. Display
4. Exit
2
The element deleted is 50

```



In this example:

1. The header files are included and a constant value 5 is defined for variable SIZE using #define statement. The SIZE defines the size of the queue.
2. A queue is created using an array named Q with an element capacity of 20. A variable named COUNT is declared to store the count of number elements present in the queue.
3. Four functions are created namely, Q_F(), Q_E(), rear_insert(), front_delete(), and display(). The user has to select an appropriate function to be performed.

4. The switch statement is used to call the rear_insert(), front_delete(), and display() functions.
5. When the user enters 1, rear_insert() function is called. In the rear_insert() function, the if loop checks if the count is full. If the condition is true, then the program prints a message "Queue is empty". Else, it checks for the value of R and assigns the element (num) entered by the user to R. Initially, when there are no elements in the queue, the value of R will be 0. After every insertion, the variable COUNT is incremented.
6. When the user enters 2, the front_delete() function is called. In this function, the if loop checks if the variable COUNT is empty. If the condition is true, then the program prints a message "Queue underflow". Else, the element in the 0th
7. When the user enters 3, the display() function is called. In this function, the if loop checks if the value of COUNT is 0. If the condition is true, the program prints a message "Queue is empty". Else, the value of F is assigned to the variable i. The for loop then displays the elements present in the queue. position will be deleted. The size of F is computed and the COUNT is set to 1.
8. When the user enters 4, the program terminates.

Priority Queue

In priority queue, the elements are inserted and deleted based on their priority. Each element is assigned a priority and the element with the highest priority is given importance and processed first. If all the elements present in the queue have the same priority, then the first element is given importance.

A priority queue is an abstract data type in which each element is associated with a priority value. Elements are served on the basis of their priority. An element with high priority is dequeued before an element with low priority. If two elements have the same priority, they are served according to their order in the queue.

The priority queue moves the highest priority elements at the beginning of the priority queue and the lowest priority elements at the back of the priority queue. It supports only those elements that are comparable. Priority queue in the data structure arranges the elements in either ascending or descending order.

Types of Priority Queue

Ascending Order Priority Queue

An ascending order priority queue gives the highest priority to the lower number in that queue



Example:

List: 5 6 20 22 10

Arrange these numbers in ascending order.

List 5 6 10 20 22

Descending Order Priority Queue

A descending order priority queue gives the highest priority to the highest number in that queue.



Example:

List: 5 6 35 22 10

Arrange these numbers

List: 35 22 10 6 5

Priority Queue Operations

Inserting an Element into the Priority Queue

Deleting an Element from the Priority Queue

Peeking from the Priority Queue (Find max/min)

Extract-Max/Min from the Priority Queue

Priority queue can be implemented using

Array

Linked list

Heap data structure

Binary search tree

Priority Queue Applications

Dijkstra's algorithm: To find shortest path in graph.

Prim's Algorithm: Prim's algorithm uses the priority queue to the values or keys of nodes and draws out the minimum of these values at every step.

Data Compression: Huffman codes use the priority queue to compress the data.

Operating Systems: load balancing and interrupt handling in an operating system

4.4 Implementation of Queues

There are two possible implementation strategies – one where the memory is used statically and the other where memory is used dynamically.

Queue can be implemented using:

Array

Linked List

Queue implementation Using Array

To represent a queue we require a one-dimensional array of some maximum size say n to hold the data items and two other variables front and rear to point to the beginning and the end of the queue.

Queue implemented using array stores only fixed number of data values. Two variables front and rear, that are implemented in queue. Front and rear variables point to the position from where insertions and deletions are performed in a queue.

Initially both front and rear are set to -1. For insert a new value into the queue, increment rear value by one and then insert at that position. For delete a value from the queue, then delete the element which is at front position and increment front value by one.

Enqueue operation

Enqueue() function is used to insert a new element into the queue. In a queue, the new element is always inserted at rear position. The enqueue() function takes one integer value as a parameter and inserts that value into the queue.

Algorithm: Enqueue operation

Step 1: IF REAR = MAX - 1

Write OVERFLOW

Go to step

[END OF IF]

Step 2: IF FRONT = -1 and REAR = -1

SET FRONT = REAR = 0

ELSE

SET REAR = REAR + 1

[END OF IF]

Step 3: Set QUEUE[REAR] = NUM

Step 4: EXIT

Deque operation

Deque() is a function used to delete an element from the queue. In a queue, the element is always deleted from front position. The Dequeue() function does not take any value as parameter.

Algorithm: Dequeue operation

Step 1: IF FRONT = -1 or FRONT > REAR

Write UNDERFLOW

ELSE

SET VAL = QUEUE[FRONT]

SET FRONT = FRONT + 1

[END OF IF]

Step 2: EXIT

Queue implementation Using Linked list

Due to the drawbacks of array. The array implementation cannot be used for the large scale applications where the queues are implemented. The alternative of array implementation is linked list implementation of queue. In a linked queue, each node of the queue consists of two parts i.e. data part and the link part. Each element of the queue points to its immediate next element in the memory.

In the linked queue, there are two pointers maintained in the memory i.e. front pointer and rear pointer. The front pointer contains the address of the starting element of the queue while the rear pointer contains the address of the last element of the queue.

Insert operation

There can be two scenarios of inserting this new node ptr into the linked queue. In the first scenario, we insert element into an empty queue. In this case, the condition front = NULL becomes true. In the second case, the queue contains more than one element. The condition front = NULL becomes false.

Algorithm

Step 1: Allocate the space for the new node PTR

Step 2: SET PTR -> DATA = VAL

Step 3: IF FRONT = NULL

SET FRONT = REAR = PTR

SET FRONT -> NEXT = REAR -> NEXT = NULL

ELSE

SET REAR -> NEXT = PTR

SET REAR = PTR

SET REAR -> NEXT = NULL

[END OF IF]

Step 4: END

Delete operation

Deletion operation removes the element that is first inserted among all the queue elements. The condition front == NULL becomes true if the list is empty. Otherwise, we will delete the element that is pointed by the pointer front.

Algorithm

Step 1: IF FRONT = NULL

Write " Underflow "

Go to Step 5

[END OF IF]

Step 2: SET PTR = FRONT

Step 3: SET FRONT = FRONT -> NEXT

Step 4: FREE PTR

Step 5: END

4.5 Applications of Queues

One major application of the queue data structure is in the computer simulation of a real-world situation. Queues are also used in many ways by the operating system, the program that schedules and allocates the resources of a computer system. One of these resources is the CPU (Central Processing Unit) itself. If you are working on a multi-user system and you tell the computer to run a particular program, the operating system adds your request to its "job queue". When your request gets to the front of the queue, the program you requested is executed. Similarly, the various users for the system must share the I/O devices (printers, disks etc.). Each device has its own queue of requests to print, read or write to these devices. The following subsection discusses one application of the queues – the priority queue. It is used in time-sharing multi-user systems where programs of high priority are processed first and programs with the same priority form a standard queue.

In Operating systems:

- a) Semaphores
- b) FCFS (first come first serve) scheduling,
- c) Spooling in printers
- d) Buffer for devices like keyboard

In Networks:

- a) Queues in routers/ switches
- b) Mail Queues

Queues are used in operating systems for handling interrupts.

Queues are used as buffers in most of the applications like MP3 media player, CD player, etc

When a resource is shared among multiple consumers.

- CPU scheduling,
- Disk Scheduling.

Summary

- A queue is an ordered collection of items in which deletion takes place at the front and insertion at the rear of the queue.
- In a memory, a queue can be represented in two ways; by representing the way in which the elements are stored in the memory, and by naming the address to which the front and rear pointers point to.
- The different types of queues are double ended queue, circular queue, and priority queue.
- The basic operations performed on a queue include inserting an element at the rear end and deleting an element at the front end.
- A priority queue is a collection of elements such that each element has been assigned a priority. An element of higher priority is processed before any element of lower priority.
- Two elements with the same priority are processed according to the order in which they were inserted into the queue.

Dictionaries

Overview

A [dictionary](#) in data structure is used to store the data in the form of key-value pair. The dictionary in data structure has an attribute called the **key**. The key is the attribute that helps us locate the data or value in the memory. A dictionary in data structure supports a wide variety of operations using a wide variety of methods and functions. The keys are always unique within a dictionary. The values of the dictionary in the data structure may or may not be unique. We can put heterogeneous type values inside the dictionaries.

As the keys are unique (no two keys can be the same in a dictionary data structure), we can easily obtain the values related to the specific **key**. Dictionary in data structures is referred to with various names such as maps, symbol tables, etc.

What is a Dictionary in Data Structure?

Before learning about the dictionary in data structure, let us first learn what is a data structure in programming and how it is related with the data types.

The data structures are nothing but a meaningful way of arranging and storing the data in the computer for efficient utilization and processing depending upon the situation. The data structure is one of the most fundamental topics of any programming language.

Apart from the data structures, we also have data types in programming languages. Data types are a classification of the data that helps us to distinguish between various categories of values. Data structures are a systematic collection of these data types. Some of the most used data types are integer, character, float, double, bytes, etc.

The data structures can be broadly divided into two categories. The first one is the built-in data structures, and the second one is user-defined data structures. Some of the most commonly used data structures are - **lists, dictionaries, tuples, sets, stacks, queues, trees, linked lists, graphs, hashmaps**, etc.

Note: Dictionary in data structure can be found as built-in data types in various programming languages like Python, C#, etc.

The dictionary in data structure is an unordered collection of data items that are used to store the data in the form of key-value pairs. The dictionary data structure has an attribute called the **key** which helps us locate the data or value in the memory. A dictionary in data structure can be compared with our regular language dictionary where a word acts like a key and its meaning(s) acts like value(s).

As we can see in a real-world dictionary, a word can have various meanings similarly, a **key** can be used to refer to various values. So, the value of a dictionary in data structure can be an immutable collection of objects or simply a single value as well.

Apart from that, the keys of a dictionary are unique and like our real-world dictionary. Since the keys are unique (no two keys can be the same in a dictionary data structure), we can easily obtain the values related to the specific **key**.

A **key** attribute of the dictionary in data structure follows two rules. Let us briefly discuss them.

1. The **key** attribute of the dictionary data structure must be unique and consist of a single element. As we have discussed above, the unique nature helps us in faster and easy retrieval of the corresponding set of values.

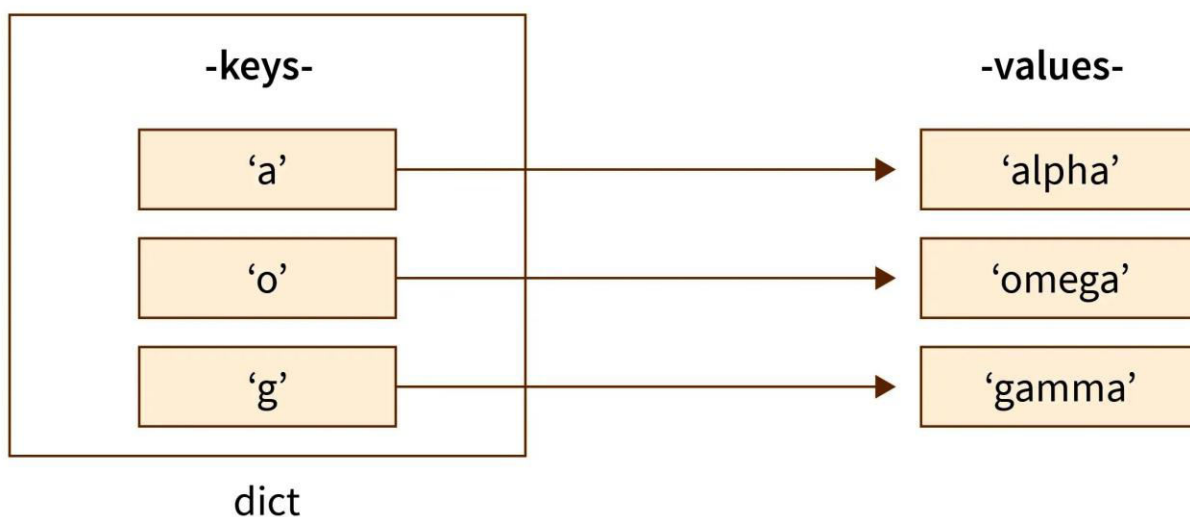
2. As no duplicate keys are allowed in the dictionary data structure, whenever a duplicate **key** is found, the last assigned value is treated as the final key-value pair. So, if we insert duplicate keys then our original data is lost.
3. The key attribute is case sensitive hence "KEY" is not the same as "key".
4. The **key** attribute is immutable hence, we cannot have an array or list as keys we can only have numbers, strings, etc, as the **key**.

The **value** part or attribute of the dictionary in data structure is simpler to define with lesser rules imposed on it. The **value** part can be a single value, or a collection of values stored in any sequential data type like lists, tuples, sets, etc.

Note: Dictionary in data structures is referred to by various names such as maps, symbol tables, etc.

The dictionary in data structure is usually created using the curly braces as {**key: value(s)**}. Now, the dictionary's key-value pair is first converted into some hash value (using a hash function). This hash code is then stored in buckets in the memory. The hash code helps in faster data retrieval as they are always unique.

Refer to the image for better visualization of the dictionary data structure.



The dictionary data structure is widely used in the field of programming (Refer to the section **Usage of the dictionary data structure in different programming languages** for detailed discussion). Since it is a widely used data structure, the programming languages have various methods or functions to support the associated operations of dictionaries in the data structure.

Let us discuss some of the most used operations of dictionary in data structure.

Operations of Dictionary in Data Structure

As we know data structures are used to store values so that they can be easily used, manipulated, and retrieved. Similarly, a dictionary in data structure is used for storing data in key-value pairs. Let us now discuss the various operation that can be performed on the dictionary in the data structure.

1. **Data insertion:** It is obvious, that every data structure stores the data inserted into it. The important point to note here is that the **key** attribute can be assigned only immutable data like numbers, strings, etc. On the other hand, the **value** attribute can store a single value, an object, or a sequence of values inside other data structures like lists, tuples, stacks, sets, etc. The data is only inserted into a dictionary data structure when there is no such key-value pair already existing in the dictionary. Now, if a key-value pair is already inserted previously then the old key-value pair will be overridden by the new key-value pair.
2. **Data updation:** The value of any data element can be easily updated. We first need to access the value using a key attribute and then update the corresponding value(s). We can access the value of a particular by using the **dictionary[key]** statement.
3. **Data deletion or removal:** We can also delete a key-value pair in a dictionary by accessing the **key** attribute. We should know that if a key is not present in the dictionary and we are trying to delete the key value then an error is shown by programming languages like Python, C#, etc.
4. **Data verification:** We can also verify or check where a particular key exists in the dictionary or not.

The data items of a dictionary are unordered. So, we can use various loops to iterate over the key-value pairs of the data. Many programming languages offer various methods for easier iteration of keys and values.

Usage of the Dictionary Data Structure in Different Programming Languages

As we know the data in the dictionary data structure is stored in key-value mappings. The dictionary in data structure internally uses hashing which makes the data retrieval faster. The hashing uses a special function called function to create hash code. As these hash codes are unique for each data (key-value) pair so we can easily retrieve data in **O(1)** time in the best cases.

The dictionary data structure can store a large set of values for a specific key. Hence it is widely used to store a heterogeneous set of data for a key. We can also create a dictionary inside another dictionary.

For example, all the data of an employee can be stored using a dictionary in data structure. Since the key needs to be unique, we can use the unique employee ID as the key and various details (such as name, designation, mobile number, address, salary, etc.) as values.

Creating a complete dictionary program in Java involves defining a data structure to store key-value pairs, implementing methods to perform basic dictionary operations (add, remove, search, and display), and creating a user interface for interaction. Below is a basic dictionary program that you can use as a starting point:

```
import java.util.HashMap;
import java.util.Map;
import java.util.Scanner;

public class DictionaryProgram {
    private Map<String, String> dictionary;

    public DictionaryProgram() {
        dictionary = new HashMap<>();
    }

    public void addWord(String word, String definition) {
        dictionary.put(word, definition);
        System.out.println("Word added to the dictionary.");
    }

    public void removeWord(String word) {
        if (dictionary.containsKey(word)) {
            dictionary.remove(word);
            System.out.println("Word removed from the dictionary.");
        } else {
            System.out.println("Word not found in the dictionary.");
        }
    }

    public void searchWord(String word) {
        if (dictionary.containsKey(word)) {
            System.out.println("Definition: " + dictionary.get(word));
        } else {
            System.out.println("Word not found in the dictionary.");
        }
    }

    public void displayDictionary() {
        for (Map.Entry<String, String> entry : dictionary.entrySet()) {
            System.out.println(entry.getKey() + ": " + entry.getValue());
        }
    }

    public static void main(String[] args) {
        DictionaryProgram dictionaryProgram = new DictionaryProgram();
        Scanner scanner = new Scanner(System.in);

        while (true) {
            System.out.println("Dictionary Program");
            System.out.println("1. Add Word");
            System.out.println("2. Remove Word");
            System.out.println("3. Search Word");
```

```
System.out.println( "4. Display Dictionary ");
System.out.println("5. Exit");
System.out.print("Enter your choice: ");
```

```
int choice = scanner.nextInt();
scanner.nextLine(); // Consume newline character
```

```
switch (choice) {
    case 1:
        System.out.print("Enter word: ");
        String word = scanner.nextLine();
        System.out.print("Enter definition: ");
        String definition = scanner.nextLine();
        dictionaryProgram.addWord(word, definition);
        break;
    case 2:
        System.out.print("Enter word to remove: ");
        String wordToRemove = scanner.nextLine();
        dictionaryProgram.removeWord(wordToRemove);
        break;
    case 3:
        System.out.print("Enter word to search: ");
        String wordToSearch = scanner.nextLine();
        dictionaryProgram.searchWord(wordToSearch);
        break;
    case 4:
        System.out.println("Dictionary:");
        dictionaryProgram.displayDictionary();
        break;
    case 5:
        System.out.println("Exiting...");
        scanner.close();
        System.exit(0);
    default:
        System.out.println("Invalid choice. Please try again.");
}
}
```

Conclusion

- The [dictionary](#) in data structure is used to store the data in the form of key-value pair.
- The key is the attribute that helps us locate the data or value in the memory.
- The keys are always unique within a dictionary. The values of the dictionary in data structure may or may not be unique.
- Dictionary in data structures are referred to by various names such as maps, symbol tables, etc.
- No duplicate keys are allowed in the dictionary data structure, so whenever a duplicate **key** is found, the last assigned value is treated as the final key-value pair.
- The key attribute is case sensitive hence "KEY" is not the same as "key".
- The **key** attribute is immutable hence, we cannot have an array or list as keys we can only have numbers, strings, etc, as the **key**.
- The **value** attribute can contain a single value or a collection of values stored in any sequential data type like lists, tuples, sets, etc.
- The dictionary in data structure internally uses hashing which makes the data retrieval faster.
- The dictionary data structure can store a large set of values for a specific key so it is widely used to store a heterogeneous set of data for a key.

Unit 12: Hashing Techniques

CONTENTS

Objectives

Introduction

12.1 Hashing

12.2 Steps to Implement Hashing

12.3 Hash Table

12.4 Hash Function

12.5 Division Method

12.6 Mid Square Method

12.7 Digit Folding Method

12.8 Multiplication Method

Summary

Keywords

Self Assessment

Answers for Self Assessment

Review Question

Further Readings

Objectives

After studying this unit, you will be able to:

- Learn basics of hashing
- Understand linear list representation
- Learn hash table representation
- Discuss hash functions and its methods

Introduction

The search time of each algorithm depend on the number n of elements of the collection S of the data. A searching technique called Hashing or Hash addressing which is essentially independent of the number n .

Hashing is the transformation of a string of characters into a usually shorter fixed-length value or key that represents the original string. Hashing is used to index and retrieve items in a database because it is faster to find the item using the shorter hashed key than to find it using the original value. It is also used in many encryption algorithms.

A Hash Function is a Unary Function that is used by Hashed Associative Containers: it maps its argument to a result of type sized. A Hash Function must be deterministic and stateless. That is, the return value must depend only on the argument, and equal arguments must yield equal results.

12.1 Hashing

In many applications we require to use a data object called symbol table. A symbol table is nothing but a set of pairs (name, value). Where value represents collection of attributes associated with

the name, and this collection of attributes depends upon the program element identified by the name. For example, if a name x is used to identify an array in a program, then the attributes associated with x are the number of dimensions, lower bound and upper bound of each dimension, and the element type. Therefore a symbol table can be thought of as a linear list of pairs (name, value), and hence you can use a list of data object for realizing a symbol table. A symbol table is referred to or accessed frequently either for adding the name, or for storing the attributes of the name, or for retrieving the attributes of the name. Therefore, accessing efficiency is a prime concern while designing a symbol table. Hence the most common way of getting a symbol table implemented is to use a hash table. Hashing is a method of directly computing the index of the table by using some suitable mathematical function called hash function. The hash function operates on the name to be stored in the symbol table, or whose attributes are to be retrieved from the symbol table. If h is a hash function and x is a name, then $h(x)$ gives the index of the table where x along with its attributes can be stored. If x is already stored in the table, then $h(x)$ gives the index of the table where it is stored to retrieve the attributes of x from the table. There are various methods of defining a hash function like a division method. In this method, you take the sum of the values of the characters, divide it by the size of the table, and take the remainder. This gives us an integer value lying in the range of 0 to $(n - 1)$ if the size of the table is n . The other method is a mid-square method. In this method, the identifier is first squared and then the appropriate number of bits from the middle of square is used as the hash value. Since the middle bits of the square usually depend on all the characters in the identifier, it is expected that different identifiers will result into different values. The number of middle bits that you select depends on the table size. Therefore if r is the number of middle bits that you use to form hash value, then the table size will be 2^r . Hence when you use this method the table size is required to be power of 2. Another method is folding in which the identifier is partitioned into several parts, all but the last part being of the same length. These parts are then added together to obtain the hash value.

To store the name or to add attributes of the name, you compute hash value of the name, and place the name or attributes as the case may be, at that place in the table whose index is the hash value of the name. For retrieving the attribute values of the name kept in the symbol table, I apply the hash function to the name to obtain index of the table where you get the attributes of the name. Hence you find that no comparisons are required to be done. Hence the time required for the retrieval is independent of the table size. Therefore, retrieval is possible in a constant amount of time, which will be the time taken for computing the hash function. Therefore, hash table seems to be the best for realization, of the symbol table, but there is one problem associated with the hashing, and it is of collisions. Hash collision occurs when the two identifiers are mapped into the same hash value. This happens because a hash function defines a mapping from a set of valid identifiers to the set of those integers, which are used as indices of the table. Therefore, you see that the domain of the mapping defined by the hash function is much larger than the range of the mapping, and hence the mapping is of many to one nature. Therefore, when I implement a hashtable a suitable collision handling mechanism is to be provided which will be activated when there is a collision.

Collision handling involves finding out an alternative location for one of the two colliding symbols. For example, if x and y are the different identifiers and if $h(x) = h(y)$, x and y are the colliding symbols. If x is encountered before y , then the i th entry of the table will be used for accommodating symbol x , but later on when y comes there is a hash collision, and therefore you have to find out an alternative location either for x or y . This means you find out a suitable alternative location and either accommodate y in that location, or you can move x to that location, and place y in the i th location of the table. There are various methods available to obtain an alternative location to handle the collision. They differ from each other in the way search is made for an alternative location. The following are the commonly used collision handling techniques.

Terminology: Hash table

Data bucket – Data buckets are memory locations where the records are stored. It is also known as unit of storage.

Hash index – It is an address of the data block. A hash function could be a simple mathematical function to even a complex mathematical function.

Linear Probing – Linear probing is a fixed interval between probes. In this method, the next available data block is used to enter the new record, instead of overwriting on the older record.

Double Hashing -Double hashing is a computer programming method used in hash tables to resolve the issues of has a collision.

Quadratic probing- It helps you to determine the new bucket address. It helps you to add Interval between probes by adding the consecutive output of quadratic polynomial to starting value given by the original computation.

Time complexity

Time complexity in linear search is $O(n)$

Time complexity in binary search is $O(\log n)$

Time complexity in hashing is $O(1)$

12.2 Steps to Implement Hashing

An element is converted into an integer by using a hash function. This element can be used as an index to store the original element, which falls into the hash table.

The element is stored in the hash table where it can be quickly retrieved using hashed key.

$\text{hash} = \text{hash func}(\text{key})$

$\text{index} = \text{hash} \% \text{array size}$

The hash is independent of the array size and it is then reduced to an index (a number between 0 and array size - 1) by using the modulo operator (%).

Operations of a hash table

Search - Searches an element in a hash table.

Insert - inserts an element in a hash table.

Delete - Deletes an element from a hash table.

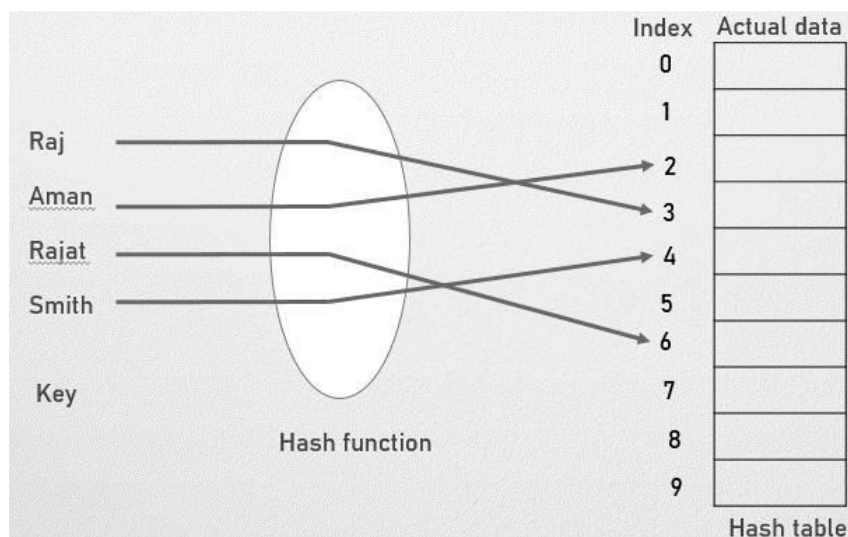
12.3 Hash Table

A Hash table is a data structure that stores information, and the information has basically two components.

- key and value.

The hash table can be implemented with the help of an associative array

It uses a hash function to compute an index into an array of buckets or slots from which the desired value can be found. It is an array of list where each list is known as bucket. It contains value based on the key. Hash table is synchronized and contains only unique elements.



12.4 Hash Function

A hash function is any function that can be used to map a data set of an arbitrary size to a data set of a fixed size, which falls into the hash table. The values returned by a hash function are called hash values, hash codes, hash sums, or hashes. An efficient hash function should be built such that the index value of the added item is distributed equally across the table.

An effective collision resolution technique should be created to generate an alternate index for a key whose hash index corresponds to a previously inserted position in a hash table.

Characteristics of hash function

Uniform Distribution: For distribution throughout the constructed table.

Fast: The generation of hash should be very fast, and should not produce any considerable overhead.

Less collisions: Collisions occur when pairs of elements are mapped to the same hash value. These should be avoided.

Some of the methods of defining hash function are discussed below:

1. **Modular arithmetic:** In this method, first the key is converted to integer, then it is divided by the size of index range, and the remainder is taken to be the hash value. The spread achieved depends very much on the modulus. If modulus is power of small integers like 2 or 10, then many keys tend to map into the same index, while other indices remain unused. The best choice for modulus is often but not always is a prime number, which usually has the effect of spreading the keys quite uniformly.

2. **Truncation:** This method ignores part of key, and use the remainder part directly as hash value. (considering non-numeric fields as their numerical code) If the keys for example are eight digit numbers and the hash table has 1000 entries, then the first, second, and fifth digit from right might make hash value. So 62538194 maps to 394. It is a fast method, but often fails to distribute keys evenly.

3. **Folding:** In this method, the identifier is partitioned into several parts all but the last part being of the same length. These parts are then added together to obtain the hash value. For example an eight digit integer can be divided into groups of three, three, and two digits. The groups are the added together, and truncated if necessary to be in the proper range of indices. Hence 62538149 maps to, $625 + 381 + 94 = 1100$, truncated to 100. Since all information in the key can affect the value of the function, folding often achieves a better spread of indices than truncation.

4. **Mid square method:** In this method, the identifier is squared (considering non-numeric fields as their numerical code), and then the appropriate number of bits from the middle depend on all the characters in the identifier, it is expected that different identifiers will result in different values. The number of middle bits that we select depends on table size. Therefore if r is the number of middle bits used to form hash value, then the table size will be 2^r , hence when you use mid square method the table size should be a power of 2.

12.5 Division Method

The hash function can be described as the hash function divides the value k by M and then uses the remainder obtained. The division method involves mapping a key k into one of m slots by taking the remainder of k divided by m as expressed in the hash function.

$$h(K) = k \bmod M$$

Here,

k is the key value, and

M is the size of the hash table.



Example:

$m=30$; $k=80$

$h(k) = k \bmod m = 20$



Example:

Hash function		Index	Actual data
$h(x) = x \% 10$	Size of hash table	0	
		1	
5		2	
10		3	
12		4	
8		5	
14		6	
19		7	
Key		8	
		9	
			Hash table

Hash function		Index	Actual data
$h(x) = x \% 10$		0	10
		1	
$5 \% 10 = 5$		2	12
$10 \% 10 = 0$		3	
$12 \% 10 = 2$		4	14
$8 \% 10 = 8$		5	5
$14 \% 10 = 4$		6	
$19 \% 10 = 9$		7	
Key		8	8
		9	19
			Hash table

Division method: pros

Any key will indeed map to one of m slots (as we want from a hash function, mapping a key to one of m slots)

Fast and simple

Division method: cons

Need to avoid certain values of m to avoid bias (as in the even number example)

12.6 Mid Square Method

In this technique, squares the key value. Then, some digits from the middle are extracted. These extracted digits form a number which is taken as the new number for address.

Limitation:

The size of key² is too large.

E.g. $2025 \times 2025 = 4100625$



Example:

$H(\text{key}) = \text{key}^2$		Index	Actual data
10	$10 \times 10 = 100$	0	10
		1	
15		2	
		3	
12		4	
		5	
22		6	
		7	
14		8	
		9	

Hash table

$H(\text{key}) = \text{key}^2$		Index	Actual data
10	$10 \times 10 = 100$	0	10
		1	
15	$15 \times 15 = 225$	2	15
		3	
12	$12 \times 12 = 144$	4	12
		5	
22	$22 \times 22 = 484$	6	
		7	
14	$14 \times 14 = 196$	8	22
		9	14

Hash table

12.7 Digit Folding Method

The folding method for constructing hash functions begins by dividing the item into equal-size pieces (the last piece may not be of equal size). These pieces are then added together to give the resulting hash value.

Fold shift

Fold boundary

-The left and right numbers are folded on fixed boundary between them and the centre.

-The two outside values are then reversed.

Case 1: if hash table size is 100 (0-99) and sum in 3 digits, then we will ignore last carry, if any.

Or

Case 2: if hash table size is 100 (0-99) and sum in 3 digits, then we need to perform the extra step of dividing by 100(size of table) and keeping the remainder.

Folding method: case 1

	Index	Actual data
Key: 122122	0	
Parts: 12 21 22	1	
		
Sum: 55		
	55	122122
Hash value: 55		
		
	99	
		Hash table

Folding method: case 2

	Index	Actual data
Key: 234567	0	
Parts: 23 45 67	1	
		
Sum: 135		
Ignore last carry	35	234567
Hash value: 35		
		
	99	
		Hash table

Folding method: case 3

Key: 234567	Index	Actual data
	0	
Parts: 23 45 67	1	
		
Sum: $135 \bmod 100 = 35$		
Sum mod with table size	35	234567
Hash value: 35		
		
	99	
		Hash table

Fold boundary: case 1

Key: 142123	Index	Actual data
	0	
Parts: 14 21 23	1	
41 21 32		
Sum: 94	94	142123
Hash value: 58		
		
	99	
		Hash table

Fold boundary: case 2

Key: 354027	Index	Actual data
	0	
Parts: 35 40 27	1	
53 40 72 value reversed		
Sum: 165		
Ignore last carry	65	354027
Hash value: 65		
		
	99	
		Hash table

12.8 Multiplication Method

$h(\text{key}) = \text{floor}(\text{table size} * (\text{key} * A)) \% \text{size}$

Or

$$h(k) = \text{floor}(n(kA \bmod 1))$$

K= key

A is constant value between 0 and 1

Knuth recommends $A = 0.6180339887...$ (Golden Ratio)

1) Choose constant

2) Multiply key k by A

3) Extract fractional part of k A (this gives us a number between 0 and 1)

4) Multiply fractional part by m and take floor of the multiplication (this transforms a number between 0 and 1, to a discrete number between 0 and m-1 that we can map to slot in the hash table)



Example: Multiplication Method

k=34

Size of table = 100

A=0.618033

$$h(34) = \text{floor}(100(34 * 0.618033)) \% 100$$

$$= \text{floor}(3461.80) \% 100$$

$$= 3461 \% 100$$

$$= 61$$

Pros: Indeed maps any key into 1 of m slots, as we expect from a hash function

Choice of m not as critical as in division method

There is also a bit implementation (not discussed)

Cons:

Slower than division method

Summary

- Hash functions are mostly used in hash tables, to quickly locate a data record (for example, a dictionary definition) given its search key (the headword).
- Specifically, the hash function is used to map the search key to the index of a slot in the table where the corresponding record is supposedly stored.
- A hash function is any function that can be used to map a data set of an arbitrary size to a data set of a fixed size, which falls into the hash table.
- The folding method for constructing hash functions begins by dividing the item into equal-size pieces (the last piece may not be of equal size).

Keywords

Hash function

Division method

Mid square method

Fold boundary

Hash table

Hashes

Modular arithmetic

Self Assessment

1. Time complexity in hashing is ____
 - A. $(\log o)$
 - B. $(\log n)$
 - C. (1)
 - D. None of above
2. The values returned by a hash function are called ____
 - A. Hash values
 - B. Hash codes
 - C. Hash sums
 - D. All of above
3. Methods for calculating the hash function are____
 - a) Division method
 - b) Folding method
 - c) Mid square method
 - d) All of above
4. Hashing is process of__
 - A. Encrypt data
 - B. Converting an input of any length into a fixed size string
 - C. Calculate mean of number
 - D. None of above